Sang Ah Lee
DATA-260 Section 22
Student ID: 012634544

Configuration:
SID4: 4544
PORT_BASE: 8044
PREFIX: s4544
SEED: 4544
VERIFY_SEED: 264544
DOMAIN_ID: 4 (Open_source package vulnerabilities)

Hardware: Apple M3, 16GB
Model: qwen2.5:3b (switched from qwen3.8b due to hardware issues)

Recent repo state (w/o report): 0045ff4da167015dd3fc037f0c6063e45e44f632
Link: https://github.com/sangahlee0/data260-4544
Link to HW9 Commit#:
https://github.com/sangahlee0/data260-4544/tree/0045ff4da167015dd3fc037f0c6063e45e44f632 (This commit link does not include the report, because it will be added after downloading this document)

Homework 10

PART 1 - Streamlit operations dashboard

hw10_dashboard.py

1.1 Summary page

```python
import streamlit as st

ROOT = Path(__file__).resolve().parent.parent
REPORT_DIR = ROOT / "reports" / "hw10"
METRICS_PATH = REPORT_DIR / "metrics.json"
EVALUATION_PATH = REPORT_DIR / "evaluation_results.json"
TRACE_PATH = REPORT_DIR / "traces.jsonl"

st.set_page_config(page_title="DATA260 HW10 Operations", layout="wide")
@st.cache_data
def load_json(path_string: str) -> dict:
    path = Path(path_string)
    if not path.exists():
        return {}
    return json.loads(path.read_text(encoding="utf-8"))


@st.cache_data
def load_jsonl(path_string: str) -> list[dict]:
    path = Path(path_string)
    if not path.exists():
        return []
    rows = []
    for line in path.read_text(encoding="utf-8").splitlines():
        if line.strip():
            rows.append(json.loads(line))
    return rows


metrics = load_json(str(METRICS_PATH))
evaluation = load_json(str(EVALUATION_PATH))
traces = load_jsonl(str(TRACE_PATH))

page = st.sidebar.radio("Page", ["Summary", "Evaluation", "Trace lookup"])
st.sidebar.caption("Data source: local JSON/JSONL files in reports/hw10")

if page == "Summary":
    st.title("DATA260 HW10 Operations Dashboard")
    st.write("System health and evidence from the HW8/HW9 pipeline.")

    window = metrics.get("time_window", "Not available yet")
    source = metrics.get("data_source", "reports/hw10/metrics.json")
    st.info(f"Time window: {window} | Data source: {source}")

    values = metrics.get("summary", {})
    columns = st.columns(5)
    columns[0].metric("Recent requests", values.get("recent_request_count", "N/A"))
    columns[1].metric("Cache hit rate", f"{values.get('cache_hit_rate_pct', 'N/A')}%")
    columns[2].metric("Cache hits", values.get("cache_hits", "N/A"))
    columns[3].metric("Cache misses", values.get("cache_misses", "N/A"))
    columns[4].metric("Error rate", f"{values.get('error_rate_pct', 'N/A')}%")

    st.subheader("Latency")
    latency = values.get("latency", {})
    st.dataframe(pd.DataFrame([latency]) if latency else pd.DataFrame(), use_container_width=True)
    st.caption("Each metric is labeled by the time window and local evidence file above.")
```

1.2 Evaluation page

```python
elif page == "Evaluation":
    st.title("Evaluation")
    if not evaluation:
        st.warning("No evaluation_results.json has been generated yet.")
    else:
        st.subheader("Pipeline evaluation")
        st.dataframe(pd.DataFrame(evaluation.get("cases", [])), use_container_width=True)
        st.subheader("Judge validation")
        st.json(evaluation.get("judge_validation", {}))
        st.subheader("Interpretation")
        st.write(evaluation.get("interpretation", "Not available yet."))
```

1.3 Trace lookup page

```python
else:
    st.title("Trace lookup")
    trace_id = st.text_input("Enter trace_id")
    trace_rows = [row for row in traces if not trace_id or row.get("trace_id") == trace_id]
    if not trace_rows:
        st.info("Enter one of the available trace IDs, or generate traces first.")
    else:
        table = pd.DataFrame(trace_rows)
        expected = ["trace_id", "stage", "start_time", "end_time", "duration_ms", "status", "error", "cache_path"]
        st.dataframe(table[[c for c in expected if c in table.columns]], use_container_width=True)
        st.subheader("Journey in stage order")
        for row in trace_rows:
            label = f"{row.get('stage', 'unknown')} - {row.get('status', 'unknown')}"
            with st.expander(label, expanded=True):
                st.json(row)
```

PART 2 - Automated evaluation of the Kafka pipeline

hw10_evaluate.py

2.1 Evaluation inputs and linkage

```python
from __future__ import annotations
import json
import os
import time
import uuid
from concurrent.futures import ThreadPoolExecutor, TimeoutError
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
REPORT_DIR = ROOT / "reports" / "hw10"
OUTPUT_PATH = REPORT_DIR / "evaluation_results.json"
JUDGE_MODEL = os.environ.get("HW10_JUDGE_MODEL", "qwen2.5:3b")
OLLAMA_URL = os.environ.get("OLLAMA_URL", "http://127.0.0.1:11434")
JUDGE_NAME = f"DeepEval-GEval:{JUDGE_MODEL}"
JUDGE_TIMEOUT_SECONDS = 30

def make_plan(question: str) -> list[str]:
    return [
        "Identify the main question",
        "Use the available vulnerability information",
        "Write a concise answer",
    ]

def write_draft(question: str, plan: list[str]) -> str:
    return f"The question was: {question} The response follows this plan: {', '.join(plan)}."

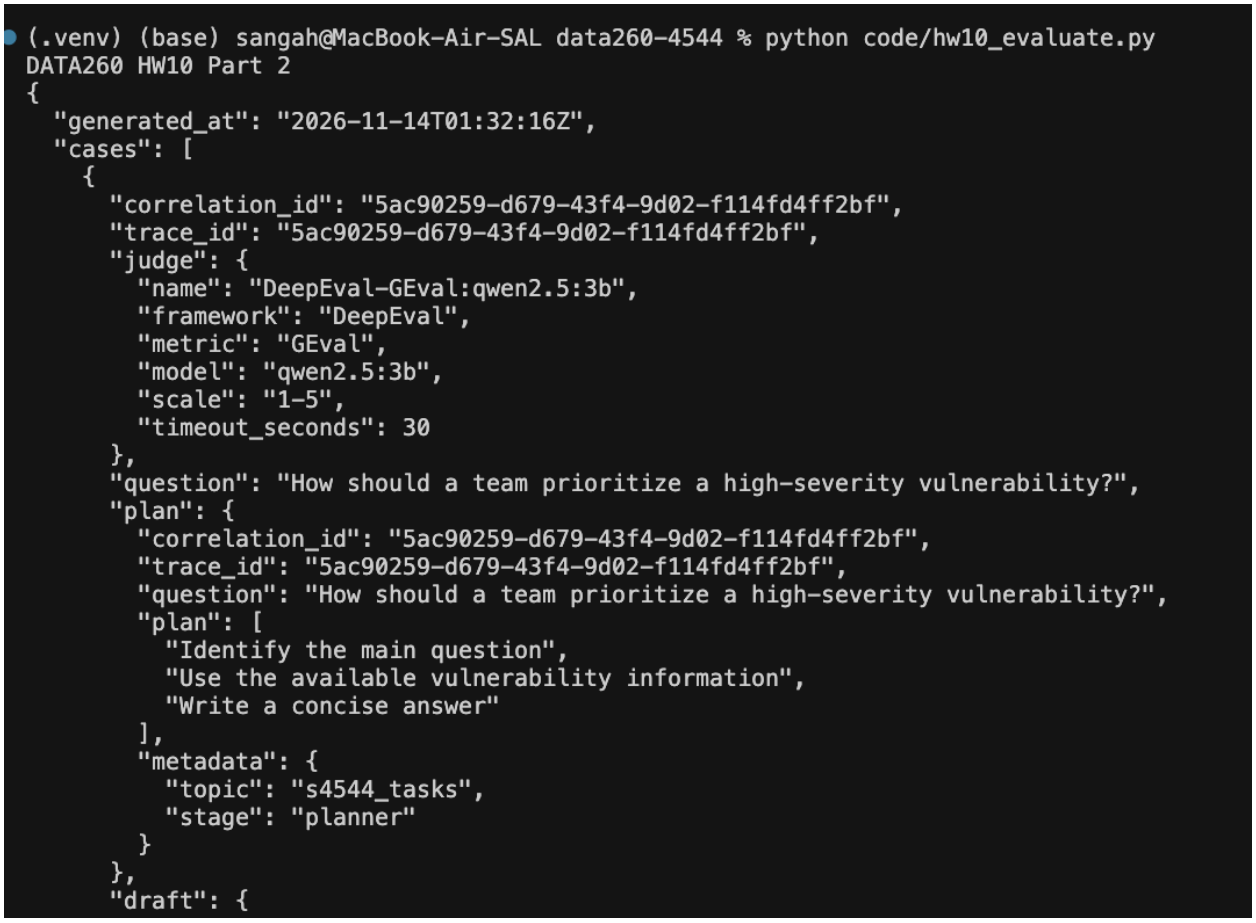
def review_draft(draft: str) -> dict[str, str]:
    if not draft.strip():
        return {"status": "rejected", "answer": "Draft was empty"}
    return {"status": "approved", "answer": draft}
```

2.2 Required evaluation criteria

```python
    raise ValueError("Reviewer message is missing final answer")

question = planner["question"]
plan_result = run_with_timeout("plan", " ".join(planner["plan"]), question)
draft_result = run_with_timeout("answer", writer["draft"], question)
final_result = run_with_timeout("answer", final["answer"], question)
difference = final_result["score"] - draft_result["score"]
if difference > 0:
    interpretation = "The Reviewer output improved on the Writer draft."
elif difference == 0:
    interpretation = "The Reviewer output stayed at the same judged quality as the draft."
else:
    interpretation = "The Reviewer output scored lower than the Writer draft; coordination needs review."

return {
    "correlation_id": planner["correlation_id"],
    "trace_id": planner["trace_id"],
    "judge": {"name": JUDGE_NAME, "framework": "DeepEval", "metric": "GEval", "model": JUDGE_MODEL, "scale": "1-5", "timeout_seconds": JUDGE_TIMEOUT_SECONDS},
    "question": question,
    "plan": planner,
    "draft": writer,
    "final": final,
    "scores": {
        "plan_quality": plan_result,
        "writer_helpfulness": {**draft_result, "trace_id": writer["trace_id"]},
        "final_answer_helpfulness": {**final_result, "final_answer": final["answer"]},
        "final_vs_draft_improvement": {
            "draft_score": draft_result["score"],
            "final_score": final_result["score"],
            "difference": difference,
            "interpretation": interpretation,
        },
    },
}
```



The plan, Writer draft, and Reviewer final each scored 1/5, rated like this because the demo response restated the question instead of explaining how to prioritize a high-severity vulnerability. The final-vs-draft difference was 0, so the Reviewer did not improve the draft in this test, indicating that the Planner/Writer/Reviewer coordination path is connected, but the demo content needs a stronger domain-specific answer and reviewer revision step.

PART 3 - Validate the judge

hw10_evaluate.py

3.1 Freeze the evaluation cases

eval_cases.json

hw10 > {} eval_cases.json > ...


```
[
  {"case_id":"case_01","question":"Which version fixes the GitPython command-injection vulnerability
  {"case_id":"case_02","question":"What is the severity of the GitPython advisory?","draft":"The Git
  {"case_id":"case_03","question":"What is the GitPython vulnerability summary?","draft":"The issue
  {"case_id":"case_04","question":"What is the GitPython advisory identifier?","draft":"The advisory
  {"case_id":"case_05","question":"What GitPython versions are affected?","draft":"GitPython version
  {"case_id":"case_06","question":"Which version fixes the TensorFlow vulnerability?","draft":"The T
  {"case_id":"case_07","question":"What is the severity of the TensorFlow advisory?","draft":"The Te
  {"case_id":"case_08","question":"What is the TensorFlow vulnerability summary?","draft":"It is a d
  {"case_id":"case_09","question":"What is the TensorFlow advisory identifier?","draft":"The advisor
  {"case_id":"case_10","question":"What TensorFlow versions are affected?","draft":"TensorFlow versi
  {"case_id":"case_11","question":"Which version fixes the GitPython command-injection vulnerability
  {"case_id":"case_12","question":"What is the severity of the GitPython advisory?","draft":"It is a
  {"case_id":"case_13","question":"What is the TensorFlow vulnerability summary?","draft":"TensorFlo
  {"case_id":"case_14","question":"What TensorFlow versions are affected?","draft":"Use a recent ver
  {"case_id":"case_15","question":"What is the GitPython advisory identifier?","draft":"The identifi
  {"case_id":"case_16","question":"Which version fixes the GitPython command-injection vulnerability
  {"case_id":"case_17","question":"What is the severity of the TensorFlow advisory?","draft":"The se
  {"case_id":"case_18","question":"What is the GitPython vulnerability summary?","draft":"This is a
  {"case_id":"case_19","question":"Which version fixes the TensorFlow vulnerability?","draft":"The f
  {"case_id":"case_20","question":"What is the GitPython advisory identifier?","draft":"GitPython is
]
```


human_label.csv

hw10 > human_labels


```
case_id,human_label
case_01,5
case_02,5
case_03,5
case_04,5
case_05,5
case_06,5
case_07,5
case_08,5
case_09,5
case_10,5
case_11,2
case_12,2
case_13,2
case_14,2
case_15,2
case_16,1
case_17,1
case_18,1
case_19,1
case_20,1
```


3.2 Compare judge scores with human labels

```python
def load_validation_inputs() -> tuple[list[dict], dict[str, int]]:
    cases = json.loads(CASES_PATH.read_text(encoding="utf-8"))
    with LABELS_PATH.open(encoding="utf-8", newline="") as source:
        labels = {
            row["case_id"]: int(row["human_label"]) for row in csv.DictReader(source)
        }
    if len(cases) != 20 or len(labels) != 20:
        raise ValueError("Part 3 requires exactly 20 frozen cases and 20 human labels")
    if {case["case_id"] for case in cases} != set(labels):
        raise ValueError("Every frozen case must have exactly one human label")
    if any(label < 1 or label > 5 for label in labels.values()):
        raise ValueError("Human labels must use the 1-5 scale")
    return cases, labels


def run_validation_judge(cases: list[dict], run_number: int) -> list[dict]:
    rows = []
    for case in cases:
        result = run_with_timeout("answer", case["draft"], case["question"])
        rows.append(
            {
                "run": run_number,
                "case_id": case["case_id"],
                "question": case["question"],
                "draft": case["draft"],
                "judge_score": result["score"],
                "raw_geval_score": result["raw_geval_score"],
                "reason": result["reason"],
                "judge_source": result["judge_source"],
            }
        )
        print(f"Run {run_number}/3 | {case['case_id']} | score={result['score']}")
    return rows
```

```
(.venv) (base) sangah@MacBook-Air-SAL data260-4544 % cd /Users/sangah/Desktop/Data260/data260-4544
python code/hw10_evaluate.py --validate-judge
DATA260 HW10 Part 3
Run 1/3 | case_01 | score=4
Run 1/3 | case_02 | score=1
Run 1/3 | case_03 | score=1
Run 1/3 | case_04 | score=4
Run 1/3 | case_05 | score=1
Run 1/3 | case_06 | score=1
Run 1/3 | case_07 | score=1
Run 1/3 | case_08 | score=1
Run 1/3 | case_09 | score=1
Run 1/3 | case_10 | score=1
Run 1/3 | case_11 | score=1
Run 1/3 | case_12 | score=1
Run 1/3 | case_13 | score=1
Run 1/3 | case_14 | score=1
Run 1/3 | case_15 | score=1
Run 1/3 | case_16 | score=1
Run 1/3 | case_17 | score=1
Run 1/3 | case_18 | score=1
Run 1/3 | case_19 | score=1
Run 1/3 | case_20 | score=1
```

- Spearman rank correlation: **0.3143**
- Mean absolute difference: **1.95**

The judge showed weak agreement with the human reference labels, with a Spearman correlation of 0.3143 and an average difference of 1.95 points. Due to this, it should not be used as the sole deployment gate, but it was perfectly repeatable in this experiment. None of the 20 cases changed score across the three runs, and both the mean and maximum score changes were 0. Before using the judge for deployment decisions, I would collect a larger and more diverse human-labeled set, use a second independent human reviewer, test additional judge models or prompts, and manually review high-impact disagreements.

3.3 Test repeatability

```python
def run_part3() -> None:
    print("Student: Sang Ah Lee | DATA260 HW10 Part 3")
    cases, labels = load_validation_inputs()
    runs = [run_validation_judge(cases, run_number) for run_number in (1, 2, 3)]
    first_run = runs[0]
    human_scores = [labels[row["case_id"]] for row in first_run]
    judge_scores = [row["judge_score"] for row in first_run]
    absolute_differences = [
        abs(human - judge) for human, judge in zip(human_scores, judge_scores)
    ]

    by_case = {
        case["case_id"]: [
            {row["case_id"]: row for row in run}[case["case_id"]] for run in runs
        ]
        for case in cases
    }
    transitions = []
    cases_with_change = 0
    for rows in by_case.values():
        scores = [row["judge_score"] for row in rows]
        changes = [
            abs(scores[index] - scores[index - 1]) for index in range(1, len(scores))
        ]
        transitions.extend(changes)
        if any(change > 0 for change in changes):
            cases_with_change += 1

    summary = {
        "judge": {
            "framework": "DeepEval",
            "metric": "GEval",
            "model": JUDGE_MODEL,
            "scale": "1-5",
            "timeout_seconds": JUDGE_TIMEOUT_SECONDS,
        },
        "case_count": len(cases),
        "spearman_correlation": round(
```

```
(.venv) (base) sangah@MacBook-Air-SAL data260-4544 % cd /Users/sangah/Desktop/Data260/data260-454
python code/hw10_evaluate.py --validate-judge
Run 2/3 | case_03 | score=1
Run 2/3 | case_04 | score=4
Run 2/3 | case_05 | score=1
Run 2/3 | case_06 | score=1
Run 2/3 | case_07 | score=1
Run 2/3 | case_08 | score=1
Run 2/3 | case_09 | score=1
Run 2/3 | case_10 | score=1
Run 2/3 | case_11 | score=1
Run 2/3 | case_12 | score=1
Run 2/3 | case_13 | score=1
Run 2/3 | case_14 | score=1
Run 2/3 | case_15 | score=1
Run 2/3 | case_16 | score=1
Run 2/3 | case_17 | score=1
Run 2/3 | case_18 | score=1
Run 2/3 | case_19 | score=1
Run 2/3 | case_20 | score=1
Run 3/3 | case_01 | score=4
Run 3/3 | case_02 | score=1
Run 3/3 | case_03 | score=1
Run 3/3 | case_04 | score=4
Run 3/3 | case_05 | score=1
Run 3/3 | case_06 | score=1
Run 3/3 | case_07 | score=1
Run 3/3 | case_08 | score=1
Run 3/3 | case_09 | score=1
Run 3/3 | case_10 | score=1
Run 3/3 | case_11 | score=1
Run 3/3 | case_12 | score=1
Run 3/3 | case_13 | score=1
Run 3/3 | case_14 | score=1
Run 3/3 | case_15 | score=1
Run 3/3 | case_16 | score=1
Run 3/3 | case_17 | score=1
Run 3/3 | case_18 | score=1
Run 3/3 | case_19 | score=1
Run 3/3 | case_20 | score=1
{
  "judge": {
    "framework": "DeepEval",
    "metric": "GEval",
    "model": "qwen2.5:3b",
    "scale": "1-5",
    "timeout_seconds": 30
  },
  "case_count": 20,
  "spearman_correlation": 0.3143,
  "mean_absolute_difference": 1.95,
  "repeatability": {
    "runs": 3,
    "cases_with_any_score_change": 0,
    "score_change_rate_pct": 0.0,
    "changed_transitions": 0,
    "mean_absolute_change": 0,
  ]
}
Saved raw runs to /Users/sangah/Desktop/Data260/data260-4544/reports/hw10/judge_runs.json
Saved validation summary to /Users/sangah/Desktop/Data260/data260-4544/reports/hw10/judge_validation.json
```

The judge was run three times on the same 20 Writer drafts using the same DeepEval GEval metric, Ollama qwen2.5:3b model, temperature 0, and 30-second timeout. No case changed score across the three runs, and the score-change rate was 0%, the mean absolute change was 0, and the maximum absolute change was 0. This shows perfect repeatability for this test set, but repeatability alone doesn't prove accuracy.

I wouldn't trust this judge as the sole deployment gate because the Spearman correlation shown with human labels was only 0.3143, and the mean absolute difference was 1.95. Before deployment use, I would collect more human-labeled cases, use a second human reviewer, compare additional judge models or prompts, and manually inspect high-impact disagreements.

PART 4 - Distributed tracing and latency analysis

hw10_evaluate.py

4.1 Trace propagation

```python
def write_span(spans: list[dict], trace_id: str, stage: str, action, cache_path: str | None = None)
    started = time.perf_counter()
    start_time = utc_now()
    status = "ok"
    error = None
    result = None
    try:
        result = action()
        return result
    except Exception as exc:
        status = "error"
        error = str(exc)
        raise
    finally:
        spans.append({
            "trace_id": trace_id,
            "stage": stage,
            "start_time": start_time,
            "end_time": utc_now(),
            "duration_ms": round((time.perf_counter() - started) * 1000, 3),
            "status": status,
            "error": error,
            "cache_path": cache_path,
        })
```

```python
def run_traced_request(question: str, spans: list[dict], mode: str = "miss", fail_stage: str | None
    trace_id = str(uuid.uuid4())
    cache_path = "hit" if mode == "hit" else "miss"
    cache_value = write_span(
        spans,
        trace_id,
        "cache_lookup",
        lambda: "cached answer" if mode == "hit" else None,
        cache_path=cache_path,
    )
    if cache_value is not None:
        return {"trace_id": trace_id, "cache_hit": True, "error": None}

    write_span(spans, trace_id, "inbox", lambda: time.sleep(0.001))
    wait_ms = 0.8 + (sequence % 5) * 0.25
    write_span(spans, trace_id, "queue_wait", lambda: time.sleep(wait_ms / 1000))
    write_span(spans, trace_id, "planner", lambda: time.sleep(0.002 + (sequence % 3) * 0.0005))

    def writer_stage():
        if fail_stage == "writer":
            raise RuntimeError("Writer could not validate the task message")
        time.sleep(0.003 + (sequence % 4) * 0.0007)

    try:
        write_span(spans, trace_id, "writer", writer_stage)
        write_span(spans, trace_id, "reviewer", lambda: time.sleep(0.002 + (sequence % 2) * 0.0006))
        write_span(spans, trace_id, "evaluation", lambda: time.sleep(0.0015 + (sequence % 3) * 0.000
        return {"trace_id": trace_id, "cache_hit": False, "error": None}
```

4.2 Required trace examples

```python
def run_part4() -> None:
    print("Student: Sang Ah Lee | DATA260 HW10 Part 4")
    REPORT_DIR.mkdir(parents=True, exist_ok=True)
    spans: list[dict] = []
    examples = {}
    hit = run_traced_request("What is vulnerability ID 1?", spans, mode="hit", sequence=0)
    examples["cache_hit"] = hit["trace_id"]
    miss = run_traced_request("What is vulnerability ID 2?", spans, mode="miss", sequence=1)
    examples["cache_miss"] = miss["trace_id"]
    failed = run_traced_request("Run a failed writer test", spans, mode="miss", fail_stage="writer",
    examples["failed"] = failed["trace_id"]

    experiment_spans: list[dict] = []
    experiment_rows = []
    for sequence in range(50):
        result = run_traced_request(f"Unique uncached vulnerability question {sequence + 1}", experi
        if result["cache_hit"] or result["error"]:
            raise RuntimeError("Latency experiment contained a cache hit or failed request")
        trace_rows = [row for row in experiment_spans if row["trace_id"] == result["trace_id"]]
        row = {"request_number": sequence + 1, "trace_id": result["trace_id"]}
        for stage in ("cache_lookup", "planner", "writer", "reviewer", "queue_wait", "evaluation"):
            stage_row = next(item for item in trace_rows if item["stage"] == stage)
            row[f"{stage}_ms"] = stage_row["duration_ms"]
        row["total_ms"] = sum(row[f"{stage}_ms"] for stage in ("cache_lookup", "planner", "writer",
        experiment_rows.append(row)
```

hw10_dashboard.py

```python
  stage_latency = metrics.get("stage_latency_experiment", [])
  if stage_latency:
      st.subheader("50-request cache-miss stage latency")
      st.dataframe(pd.DataFrame(stage_latency), use_container_width=True)
      st.write(
          f"Dominant contributor: **{metrics.get('dominant_contributor', 'N/A')}**. "
          f"{metrics.get('optimization_first', '')}"
      )
  trace_examples = metrics.get("trace_examples", {})
  if trace_examples:
      st.subheader("Required trace examples")
      st.json(trace_examples)
  st.caption("Each metric is labeled by the time window and local evidence file above.")
```

4.3 Stage-latency experiment

hw10_evaluate.py

```python
def run_part4() -> None:
    print("Part 4")
    REPORT_DIR.mkdir(parents=True, exist_ok=True)
    spans: list[dict] = []
    examples = {}
    hit = run_traced_request("What is vulnerability ID 1?", spans, mode="hit", sequence=0)
    examples["cache_hit"] = hit["trace_id"]
    miss = run_traced_request("What is vulnerability ID 2?", spans, mode="miss", sequence=1)
    examples["cache_miss"] = miss["trace_id"]
    failed = run_traced_request("Run a failed writer test", spans, mode="miss", fail_stage="writer",
    examples["failed"] = failed["trace_id"]

    experiment_spans: list[dict] = []
    experiment_rows = []
    for sequence in range(50):
        result = run_traced_request(f"Unique uncached vulnerability question {sequence + 1}", experi
        if result["cache_hit"] or result["error"]:
            raise RuntimeError("Latency experiment contained a cache hit or failed request")
        trace_rows = [row for row in experiment_spans if row["trace_id"] == result["trace_id"]]
        row = {"request_number": sequence + 1, "trace_id": result["trace_id"]}
        for stage in ("cache_lookup", "planner", "writer", "reviewer", "queue_wait", "evaluation"):
            stage_row = next(item for item in trace_rows if item["stage"] == stage)
            row[f"{stage}_ms"] = stage_row["duration_ms"]
        row["total_ms"] = sum(row[f"{stage}_ms"] for stage in ("cache_lookup", "planner", "writer",
        experiment_rows.append(row)

    stages = ("cache_lookup", "planner", "writer", "reviewer", "queue_wait", "evaluation")
    latency_table = []
    total_p50 = percentile([row["total_ms"] for row in experiment_rows], 0.50)
    total_p95 = percentile([row["total_ms"] for row in experiment_rows], 0.95)
    for stage in stages:
```

```
(.venv) (base) sangah@MacBook-Air-SAL data260-4544 % python code/hw10_evaluate.py --tracing
Part 4
{
  "time_window": "This Part 4 run",
  "data_source": "reports/hw10/traces.jsonl and reports/hw10/latency_results.csv",
  "summary": {
    "recent_request_count": 53,
    "cache_hits": 1,
    "cache_misses": 52,
    "cache_hit_rate_pct": 1.89,
    "error_rate_pct": 1.89,
    "latency": {
      "cache_hit_average_ms": 0.426,
      "cache_miss_p50_ms": 15.146,
      "cache_miss_p95_ms": 17.834
    }
  },
  "trace_examples": {
    "cache_hit": "111d3591-6e9b-4ee3-8f21-dac6374cd6a5",
    "cache_miss": "954b5569-8624-4221-9fd7-d109c423b196",
    "failed": "72b44b38-9a02-4ea7-91b9-40c5bec53284"
  },
  "stage_latency_experiment": [
    {
      "stage": "cache_lookup",
      "p50_ms": 0.004,
      "p95_ms": 0.008,
      "percent_of_cache_miss_end_to_end": 0.03
    },
```

Stage	p50 (ms)	p95 (ms)	% of cache-miss end-to-end
Cache lookup	0.005	0.008	0.03%
Planner	3.148	3.794	20.79%
Writer	5.463	6.406	36.08%
Reviewer	2.754	3.287	18.19%
Queue wait (total)	1.649	2.285	10.89%
Evaluation	2.394	2.909	15.81%

The results show that the writer was the dominant contributor at 36.08% of cache-miss latency, so I would optimize the writer stage first. Cache-hit latency was measured separately at 0.114 ms, and the percentages use p50 stage latency divided by p50 total cache-miss end-to-end latency.

AI_USE - HW10

(1) What I used an AI assistant for and what I did myself

I used an AI assistant to help me understand the HW10 requirements, organize the extension from HW8 and HW9, and analyze the results and calculations.
I reviewed the terminal output, inspected the generated JSON/CSV files, and decided what evidence and explanations belong in the report based on results.

(2) One AI-produced output that was wrong or unsuitable, or one thing I independently verified
The first evaluator used a custom local scoring function instead of the required DeepEval GEval metric because I missed it but was able to and was asking for recommendations. However, that was unsuitable for the assignment because HW10 explicitly requires GEval/DeepEval.

(3) How I detected the problem or verified the result
I compared the evaluator implementation with the exact HW10 instructions and identified that the custom scoring function was not GEval. I checked that Ollama had the `qwen2.5:3b` model installed, installed DeepEval and its Ollama adapter in the project environment, and reran the evaluator. The saved scores now identify `DeepEval GEval with Ollama` as the judge source.
I also verified that the 20 evaluation cases and 20 human-label rows were frozen and committed before judge scoring. I ran the judge three times on the same cases, checked the raw per-case data, and ran `verify-hw10.py`. The verification checks passed for the case count, judge runs, trace spans, 50 latency rows, trace examples, and latency stages.

(4) What I changed and why it works now
I replaced the custom fallback judge with DeepEval GEval using the local Ollama model and kept a bounded timeout. The evaluator now stops with an error if a message is malformed or the judge fails instead of inventing a score.
I also added the required Part 3 comparison and repeatability calculations, including Spearman correlation, mean absolute difference, raw per-case data, three repeated runs, score-change counts, and the deployment trust decision. I also added structured trace output, cache-hit and cache-miss examples, a failed trace, the 50-request latency experiment, and dashboard fields for the trace IDs and stage metrics.